

Travelling Salesman Problem with Time Windows

Group 3

Member 1: Le Khanh Bao Minh 202516596
Member 2: Pham Dinh Nhat Minh 202516602
Member 3: Nguyen Huy Tu 202516684
Member 4: Nguyen Phu Cuong 202516647

Course Information

Class ID: 167686
Professor: Bui Quoc Trung

Abstract

This report investigates the Travelling Salesman Problem with Time Windows (TSPTW), focusing on optimizing delivery routes from a central warehouse to multiple customers. The objective is to determine a delivery sequence that minimizes total travel time while strictly adhering to customer-specific time windows and service durations. To address this problem, the study evaluates a wide spectrum of algorithms, including exact methods (Branch and Bound, Constraint Programming, and Mixed Integer Programming), a two-phase heuristic approach (Greedy and Local Search), and advanced metaheuristics (Ant Colony Optimization and Adaptive Large Neighborhood Search). Experimental evaluations across both tight and loose time-window datasets reveal distinct performance trade-offs. Exact solvers such as Branch and Bound and MIP succumb to combinatorial explosion on larger datasets, whereas Constraint Programming reliably guarantees optimal solutions up to 1,000 nodes, albeit with high computational time costs. Conversely, the two-phase heuristic approach, particularly Local Search, demonstrates an exceptional balance, delivering near-optimal routes in significantly less time, while Adaptive Large Neighborhood Search shows strong time scalability. Ultimately, these findings provide useful insights for selecting appropriate routing algorithms based on the specific time constraints and scale of real-world logistics scenarios.

Contents

1	Introduction to TSPTW	3
2	Exact Algorithms	4
2.1	Branch and Bound	4
2.2	Constraint Programming	6
2.3	Mixed Integer Programming	7
3	Heuristic Algorithms	8
3.1	Greedy Algorithm	8
3.2	Local Search Algorithm (Hill Climbing)	10
4	Metaheuristic Algorithms	11
4.1	Ant Colony Optimization (ACO)	11
4.2	Adaptive Large Neighborhood Search (ALNS)	14
5	Results and Analysis	17
5.1	Hustack test cases (Tight time windows)	17
5.2	Code-generated test cases (Loose time windows)	19
6	Conclusion	23
6.1	Strengths and Limitations of Each Algorithm	23
6.2	Future Research Directions	23
7	Appendix	24
7.1	Appendix 1. Derivation of $M_{i,j}$	24
7.2	Appendix 2. Explanation of MTZ constraint	25
7.3	Appendix 3. Derivation of $\tau_{\max}$	25
7.4	Appendix 4. Derivation of T	26

1. Introduction to TSPTW

A delivery worker starts from a warehouse (point 0) and must deliver goods to N customers located at points 1 through N . Each customer i has a delivery time window from e_i to l_i , and each delivery takes d_i units of time. The travel time between any two points i and j is given as $c_{i,j}$. The delivery worker departs from the warehouse at time t_0 . The objective is to compute a delivery route that minimises the total travel time while ensuring that all deliveries are made within their respective time windows. Each solution is represented by a permutation $s[1], s[2], \dots, s[N]$ of the customer indices 1 through N , indicating the order in which the customers are visited.

The data format:

- Line 1: contains a positive integer N ($1 \leq N \leq 1000$).
- Line $i + 1$ ($i = 1, \dots, N$): contains e_i , l_i , and d_i .
- Line $i + N + 2$ ($i = 0, 1, \dots, N$): contains the i -th row of the matrix c .

Test cases by Hustack:

- N is chosen from $\{5, 10, 100, 200, 300, 500, 600, 700, 900, 1000\}$.
- The service time d_i for each customer i ranges from 5 to 20.
- The travel time $t_{i,j}$ between any two points is less than 1000.

Code-generated test cases:

- 22 test cases with $N \in \{5, 10, 50, 100, 150, \dots, 1000\}$

2. Exact Algorithms

2.1 Branch and Bound

Branch and Bound is an exact algorithm that systematically enumerates candidate solutions by exploring a state-space search tree via recursive backtracking. It guarantees finding the optimal route by evaluating all feasible permutations while actively pruning branches that cannot possibly yield a better solution than the currently known best sequence.

- N : number of customers
 - $c_{i,j}$: the travel time from point i to point j
 - e_i, l_i, d_i : the earliest time, latest time, and service duration at point i
 - $c_{\min}$: the minimum travel cost between any two distinct nodes in the entire network
 - k : the length of the current route
- (1) Before exploring the search space, the algorithm computes the global minimum edge cost $c_{\min}$ across the entire graph to serve as a baseline for optimistic future cost predictions.
 - (2) From the current node, the algorithm generates branches to unvisited candidate nodes, only considers those whose projected arrival time is sooner than the deadline.

$$e_i + c_{i,j} + d_i \leq l_j$$

- (3) The algorithm computes a lower bound (LB) by adding the current total cost with an optimistic projection of the remaining journey. If the $LB \geq C_{best}$, the branch is pruned.

$$LB = \text{current cost} + (N - k) \cdot c_{\min}$$

- (4) If the branch is promising, the search continues recursively depth-first. Once a sequence successfully visits all customers and returns to the depot, the global best cost is updated.
- (5) The algorithm terminates once it has exhausted all possible paths.

Algorithm 1: Branch and Bound

Input: N, e, l, d, c **Output:** Optimal feasible route and cost

```
1 Function BranchAndBound( $N, e, l, d, c$ ):
2    $c_{\min} \leftarrow \min_{i \neq j} c_{i,j}$ 
3    $\text{best\_cost} \leftarrow \infty, \text{best\_route} \leftarrow []$ 
4    $\text{visited}[1 \dots N] \leftarrow \text{False}$ 
5    $\text{route} \leftarrow []$ 
6   Function Backtrack( $\text{level}, \text{curr}, \text{time}, \text{cost}$ ):
7     if  $\text{cost} + (N - \text{level}) \cdot c_{\min} \geq \text{best\_cost}$  then
8       return
9     if  $\text{level} = N$  then
10       $\text{total\_cost} \leftarrow \text{cost} + c_{\text{curr},0}$ 
11      if  $\text{total\_cost} < \text{best\_cost}$  then
12         $\text{best\_cost} \leftarrow \text{total\_cost}$ 
13         $\text{best\_route} \leftarrow \text{route}$ 
14      return
15     for  $\text{nxt} \leftarrow 1$  to  $N$  do
16       if not  $\text{visited}[\text{nxt}]$  then
17         if feasible then
18            $\text{finish} \leftarrow \max(\text{time} + c_{\text{curr},\text{nxt}} + d_{\text{curr},e_{\text{nxt}}}) + d_{\text{nxt}}$ 
19            $\text{visited}[\text{nxt}] \leftarrow \text{True}$ 
20           Add  $\text{nxt}$  to route
21           Backtrack( $\text{level} + 1, \text{nxt}, \text{finish}, \text{cost} + c_{\text{curr},\text{nxt}}$ )
22           Remove  $\text{nxt}$  from route
23            $\text{visited}[\text{nxt}] \leftarrow \text{False}$ 
24   Backtrack( $0, 0, 0, 0$ )
25   return  $\text{best\_route}$ 
```

2.2 Constraint Programming

- N : number of customers
- $c_{i,j}$: the travel time from point i to point j
- e_i, l_i, d_i : the earliest time, latest time, and service duration at point $i \in \{1, 2, \dots, N\}$
Note: we let $e_0 = 0, l_0 = \max_{i \in \{1, \dots, N\}}(l_i + d_i + c_{i,0}), d_0 = 0$

2.2.1 Decision Variables

- $x_{i,j} \in \{0, 1\}$: 1 if the route goes directly from i to j , 0 otherwise
- $t_i \in \mathbb{N}$: arrival time at point i

2.2.2 Constraints

- **Graph sparsification**

$$A = \{(i, j) \in \{0, 1, \dots, N\}^2 \mid i \neq j \text{ and } (j = 0 \vee e_i + c_{i,j} + d_i \leq l_j)\}$$

We only consider arcs from i to j ($i \neq j$) if it satisfies one out of two following conditions:

- (1) $j = 0$: It is always possible to return to the depot (point 0).
- (2) $e_i + c_{i,j} + d_i \leq l_j$: The earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$) does not violate the deadline of j (l_j).

- **Hamiltonian circuit constraint**

$$\text{Circuit}(\{x_{i,j} \mid (i, j) \in A\})$$

The arcs must form a Hamiltonian cycle.

- **Time constraints**

$$e_i \leq t_i \leq l_i \quad \forall i = 1, \dots, N$$

The arrival time at point i must belong to the window $[e_i, l_i]$.

$$t_j \geq t_i + d_i + c_{i,j} \quad \forall (i, j) \in A \text{ if } x_{i,j} = 1 \text{ and } j \neq 0$$

The arrival time at point j must be greater than or equal to the earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$).

2.2.3 Objective Function

$$\min \sum_{(i,j) \in A} c_{i,j} \cdot x_{i,j}$$

2.3 Mixed Integer Programming

- N : number of customers
- $c_{i,j}$: the travel time from point i to point j
- e_i, l_i, d_i : the earliest time, latest time, and service duration at point $i \in \{1, 2, \dots, N\}$
Note: we let $e_0 = 0, l_0 = \max_{i \in \{1, \dots, N\}}(l_i + d_i + c_{i,0}), d_0 = 0$

2.3.1 Decision Variables

- $x_{i,j} \in \{0, 1\}$: 1 if the route goes from i to j , 0 otherwise
- $t_i \in \mathbb{R}_{\geq 0}$: arrival time at customer i
- $u_i \in \{1, \dots, N\}$: sequencing variable for subtour elimination

2.3.2 Constraints

- **Graph sparsification**

$$A = \{(i, j) \in \{0, 1, \dots, N\}^2 \mid i \neq j \text{ and } (j = 0 \vee e_i + c_{i,j} + d_i \leq l_j)\}$$

We only consider arcs from i to j ($i \neq j$) if it satisfies one out of two following conditions:

- (1) $j = 0$: It is always possible to return to the depot (point 0).
- (2) $e_i + c_{i,j} + d_i \leq l_j$: The earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$) does not violate the deadline of j (l_j).

- **Flow conservation**

$$\sum_{j \mid (i,j) \in A} x_{i,j} = 1 \quad \forall i \in \{0, 1, \dots, N\}$$

$$\sum_{i \mid (i,j) \in A} x_{i,j} = 1 \quad \forall j \in \{0, 1, \dots, N\}$$

Each point i has exactly one outgoing arc, and each point j has exactly one incoming arc.

- **Time constraints**

$$e_i \leq t_i \leq l_i \quad \forall i = 1, \dots, N$$

The arrival time at point i must belong to the window $[e_i, l_i]$.

$$t_j \geq t_i + d_i + c_{i,j} - M_{i,j}(1 - x_{i,j}) \quad \forall (i, j) \in A \text{ and } j \neq 0$$

where

$$M_{i,j} = \max(0, l_i - e_j + d_i + c_{i,j})$$

The arrival time at point j must be greater than or equal to the earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$). Derivation of $M_{i,j}$ can be found in Appendix 1.

- **Subtour elimination (MTZ constraint)**

$$u_i - u_j + N \cdot x_{i,j} \leq N - 1 \quad \forall (i, j) \in A \text{ with } i \neq 0 \text{ and } j \neq 0$$

Further details about the MTZ constraint can be found in Appendix 2.

2.3.3 Objective Function

$$\min \sum_{(i,j) \in A} c_{i,j} \cdot x_{i,j}$$

3. Heuristic Algorithms

To balance computational efficiency with high solution quality, a **two-phase heuristic** approach is used. Because exact solvers (like MIP and CP) struggle with exponentially growing search spaces, this heuristic relies on a rapid **Greedy Phase** to find an initial feasible route, followed by a **Local Search Phase** to iteratively refine the objective cost.

3.1 Greedy Algorithm

The first phase uses a greedy algorithm to quickly generate a valid starting solution. Because time-window constraints make the problem very strict, short-sighted rules can easily lead to suboptimal routes or dead ends. To ensure a solid baseline, a **Combined Heuristic** approach is used, building a single route that balances travel cost with urgency.

- At any current node i , a candidate node j is considered feasible only if the vehicle can arrive before its deadline. Let $\text{arrival}_j = t + d_i + c_{i,j}$, the condition is: $\text{arrival}_j \leq l_j$.
- For each feasible candidate j , the algorithm looks at these details:
 - **Start Time:** $\text{start}_j = \max(\text{arrival}_j, e_j)$
 - **Finish Time:** $\text{finish}_j = \text{start}_j + d_j$
 - **Waiting Time:** $\text{wait}_j = \text{start}_j - \text{arrival}_j$
 - **Slack:** $\text{slack}_j = l_j - \text{arrival}_j$
- The algorithm builds the sequence by greedily picking the candidate j that minimizes the combined score H . This score looks at travel time, waiting time, and how close the deadline is:

$$H = c_{i,j} + \text{wait}_j + 0.35 \cdot \text{slack}_j$$

$$\min_{j \in \text{candidates}} H$$

- By using $(l_j - \text{arrival}_j)$ for slack instead of $(l_j - \text{start}_j)$, the formula stops the wait time and slack values from canceling each other out in the math. This ensures the algorithm still avoids long waits while rushing to customers with approaching deadlines.
- Once the complete route is built, it is passed as the initial route into the Local Search phase.
- If at any point no valid candidate is found, the route is a dead end and the algorithm returns an empty route.

Algorithm 2: Greedy Heuristic

Input: N, e, l, d, c **Output:** Feasible route or empty list if infeasible

```
1 Function Greedy( $N, e, l, d, c$ ):
2   visited[1... $N$ ]  $\leftarrow$  0
3   route  $\leftarrow$  []
4   time  $\leftarrow$  0,  $i \leftarrow$  0
5   while |route| <  $N$  do
6     candidates  $\leftarrow$  []
7     for  $j \leftarrow$  1 to  $N$  do
8       if visited[ $j$ ] = 0 then
9         arrival  $\leftarrow$  time +  $d_i + c_{i,j}$ 
10        if arrival  $\leq l_j$  then
11          start  $\leftarrow$  max(arrival,  $e_j$ )
12          finish  $\leftarrow$  start +  $d_j$ 
13          wait  $\leftarrow$  start - arrival
14          slack  $\leftarrow$   $l_j$  - arrival
15           $H \leftarrow c_{i,j} +$  wait +  $0.35 \cdot$  slack
16          Add ( $H, j, \text{start}, \text{finish}$ ) to candidates
17        if candidates is empty then
18          return [],  $\infty$ 
19        Choose ( $H^*, j^*, s^*, f^*$ )  $\in$  candidates with minimum  $H^*$ 
20        Update time  $\leftarrow s^*$ ,  $i \leftarrow j^*$ 
21        Add  $j^*$  to route, update visited[ $j^*$ ]  $\leftarrow$  1
22  return route
```

3.2 Local Search Algorithm (Hill Climbing)

To refine the initial suboptimal route of the Greedy construction, a **Local Search** algorithm is applied.

- (1) For an initial route, the algorithm generates neighboring solutions by iterating over all possible pairs of nodes in the sequence and swapping their positions.
- (2) The algorithm uses a **First Improvement** rule. It immediately accepts the first valid swap that results in a temporally feasible route with a strictly lower cost, halting the current neighborhood search.
- (3) The search then restarts from step (1), using this newly improved route as the baseline to explore its neighbors.
- (4) The algorithm terminates when it reaches a **Local Minimum**—a state where no further feasible improvements are found.

Algorithm 3: Local Search

```

Input:  $N, e, l, d, c$ 
Output: Improved feasible route
1 Function LocalSearch( $N, e, l, d, c$ ):
2    $\text{best\_route}, \text{best\_cost} \leftarrow \text{Greedy}(N, e, l, d, c)$ 
3    $\text{improving} \leftarrow \text{True}$ 
4   while improving do
5      $\text{improving} \leftarrow \text{False}$ 
6     Cache the timeline and accumulated costs of  $\text{best\_route}$ 
7     for  $i = 0$  to  $N - 2$  do
8       for  $j = i + 1$  to  $N - 1$  do
9          $\text{neighbor} \leftarrow \text{Swap nodes } i \text{ and } j \text{ in a copy of } \text{best\_route}$ 
10        // Fast check: Only simulate the route from index  $i$  onwards
11         $\text{feasible}, \text{new\_cost} \leftarrow \text{Evaluate}(\text{neighbor}, i)$ 
12        if  $\text{feasible and new\_cost} < \text{best\_cost}$  then
13           $\text{best\_route} \leftarrow \text{neighbor}$ 
14           $\text{best\_cost} \leftarrow \text{new\_cost}$ 
15           $\text{improving} \leftarrow \text{True}$ 
16          break // First-improvement strategy
17        if improving then
18          break
19   return  $\text{best\_route}, \text{best\_cost}$ 

```

4. Metaheuristic Algorithms

While the two-phase heuristic approach provides computationally fast and feasible initial routes, local search mechanisms are inherently prone to getting trapped in local optima, particularly due to the highly rugged search space created by strict time-window constraints. To achieve near-optimal solutions for larger instances, more sophisticated exploration strategies are required.

This section introduces two advanced metaheuristic frameworks employed to overcome these limitations: **Ant Colony Optimization (ACO)** and **Adaptive Large Neighborhood Search (ALNS)**. The following subsections detail the mathematical formulation and algorithmic implementation of both approaches.

4.1 Ant Colony Optimization (ACO)

Ant Colony Optimization is a population-based metaheuristic inspired by the foraging behavior of real ants. In the context of the Travelling Salesman Problem with Time Windows (TSPTW), artificial ants iteratively construct delivery routes. They make probabilistic routing decisions based on two factors: **pheromone trails** (representing the historical success of previous ants on a path) and **heuristic information** (representing the immediate, greedy desirability of visiting a node). Over multiple iterations, the pheromone trails are updated to reinforce shorter, feasible routes.

- N : number of customers
- $c_{i,j}$: the travel time from point i to point j
- e_i, l_i, d_i : the earliest time, latest time, and service duration at point i
- α : the relative importance of the pheromone trail
- β : the relative importance of the heuristic information
- ρ : the pheromone evaporation rate
- Q : the pheromone deposit factor

4.1.1 Graph Sparsification & Feasibility

Similar to the exact models, ants only consider feasible transitions. From a current node i , an unvisited candidate node j is only added to the allowed set if the ant can arrive before the deadline:

$$A_i = \{j \in \{1, \dots, N\} \mid \text{visited}_j = 0 \text{ and } t + d_i + c_{i,j} \leq l_j\}$$

If A_i is empty before all customers are visited, the ant has reached a dead end and the route is discarded.

4.1.2 Heuristic Information

The heuristic information $\eta_{i,j}$ guides ants toward nodes that make logical sense based on time constraints and travel cost. It relies on the combined heuristic score $H_{i,j}$ defined previously in the Greedy phase. To convert this cost into a desirability metric (where higher is better), it is inverted:

$$\eta_{i,j} = \frac{1}{H_{i,j} + \epsilon}$$

where ϵ is a small constant used to prevent division by zero.

4.1.3 State Transition Rule

An ant at node i chooses the next node $j \in A_i$ based on the probability $P_{i,j}$:

$$P_{i,j} = \frac{[\tau_{i,j}]^\alpha \cdot [\eta_{i,j}]^\beta}{\sum_{k \in A_i} [\tau_{i,k}]^\alpha \cdot [\eta_{i,k}]^\beta}$$

where $\tau_{i,j}$ is the current amount of pheromone on the arc (i, j) . This equation ensures a balance between exploration (picking edges with high heuristic value) and exploitation (following historically successful paths).

4.1.4 Pheromone Update Rules

To prevent the algorithm from converging instantly on a suboptimal route, a **Min-Max Ant System (MMAS)** approach is used alongside an elitist update strategy.

- (1) **Evaporation:** At the end of each iteration, pheromones on all edges evaporate to slowly forget older, potentially poor routes:

$$\tau_{i,j} \leftarrow (1 - \rho) \cdot \tau_{i,j}$$

- (2) **Elitist Deposit:** Only the ant that found the best route in the current iteration is allowed to deposit pheromone, aggressively reinforcing the most successful sequence found so far. The update rule is defined as:

$$\tau_{i,j} \leftarrow \tau_{i,j} + \Delta\tau_{i,j}$$

where the deposited pheromone $\Delta\tau_{i,j}$ is calculated piecewise:

$$\Delta\tau_{i,j} = \begin{cases} \frac{Q}{L_{\text{best}}} & \text{if } (i, j) \in \text{Best Route} \\ 0 & \text{otherwise} \end{cases}$$

and L_{best} represents the total travel cost of that iteration's best route.

- (3) **Min-Max Bounds:** To maintain exploration and prevent the probabilities from locking into a single path, all pheromone values are strictly bounded within a dynamic range $[\tau_{\min}, \tau_{\max}]$. These bounds are updated based on the global best route cost ($L_{\text{global_best}}$):

$$\tau_{\max} = \frac{Q}{\rho \cdot L_{\text{global_best}}}$$

$$\tau_{\min} = \frac{1}{N} \cdot \tau_{\max}$$

After the elitist deposit, the pheromone trails are clamped to respect these limits:

$$\tau_{i,j} \leftarrow \max(\tau_{\min}, \min(\tau_{\max}, \tau_{i,j}))$$

For details on the derivation of $\tau_{\max}$, see Appendix 3.

Algorithm 4: Ant Colony Optimization (ACO)

Input: $N, e, l, d, c, \alpha, \beta, \rho, Q, \max_no_imp$
Output: Improved feasible route

```

1 Function ACO( $N, e, l, d, c, \alpha, \beta, \rho, Q, \max\_no\_imp$ ):
2    $best\_route, best\_cost \leftarrow Greedy(N, e, l, d, c)$ 
3   Calculate initial bounds  $\tau_{max}, \tau_{min}$  using  $best\_cost$ 
4   Initialize pheromone matrix  $\tau_{i,j} \leftarrow \tau_{max}$  for all  $i \neq j$ 
5   if  $best\_cost < \infty$  then
6     Set the pheromone of edges in  $best\_route$  with  $\tau_{max}$ 
7     Set the pheromone of the remaining edges to  $\tau_{min}$ 
8    $m \leftarrow \min(50, N)$  // Number of ants
9    $not\_improved \leftarrow 0$ 
10  while  $not\_improved < \max\_no\_imp$  do
11     $iter\_best\_route \leftarrow []$ 
12     $iter\_best\_cost \leftarrow \infty$ 
13    for  $k \leftarrow 1$  to  $m$  do
14       $visited[1 \dots N] \leftarrow 0, visited[0] \leftarrow 1$ 
15       $route \leftarrow [0], i \leftarrow 0, time \leftarrow 0, L \leftarrow 0$ 
16      for  $step \leftarrow 1$  to  $N$  do
17         $A_i \leftarrow \{j \mid visited[j] = 0 \text{ and } time + d_i + c_{i,j} \leq l_j\}$ 
18        if  $A_i$  is empty then
19          break
20        for  $j \in A_i$  do
21          Calculate  $H_{i,j}$  and desirability  $\eta_{i,j}$ 
22          Calculate transition probability  $P_{i,j}$ 
23        Select next node  $j$  from  $A_i$  using Roulette Wheel based on  $P_{i,j}$ 
24         $time \leftarrow \max(time + d_i + c_{i,j}, e_j)$ 
25         $L \leftarrow L + c_{i,j}$ 
26        Add  $j$  to  $route$ , update  $visited[j] \leftarrow 1, i \leftarrow j$ 
27      if  $|route| = N + 1$  then
28         $L \leftarrow L + c_{i,0}$  // Return to depot
29        Add 0 to  $route$ 
30        if  $L < iter\_best\_cost$  then
31           $iter\_best\_route \leftarrow route$ 
32           $iter\_best\_cost \leftarrow L$ 
33        if  $L < best\_cost$  then
34           $best\_route \leftarrow route$ 
35           $best\_cost \leftarrow L$ 
36           $not\_improved \leftarrow -1$  // Will be incremented to 0 below
37          Update  $\tau_{max}, \tau_{min}$  using new  $best\_cost$ 
38       $\tau_{i,j} \leftarrow (1 - \rho) \cdot \tau_{i,j} \quad \forall i, j$ 
39      if  $iter\_best\_cost < \infty$  then
40         $\tau_{i,j} \leftarrow \tau_{i,j} + \frac{Q}{iter\_best\_cost} \quad \forall (i, j) \in iter\_best\_route$  // Elitist deposit
41       $\tau_{i,j} \leftarrow \max(\tau_{min}, \min(\tau_{max}, \tau_{i,j}))$ 
42       $not\_improved \leftarrow not\_improved + 1$ 
43  return  $best\_route$  excluding depot nodes

```

4.2 Adaptive Large Neighborhood Search (ALNS)

Adaptive Large Neighborhood Search is a metaheuristic that iteratively improves a solution by partially destroying and subsequently repairing it. Compared to Local Search, ALNS explores significantly larger search spaces, allowing the algorithm to escape local optima more effectively. In the context of TSPTW, the algorithm relies on a competitive pool of destroy and repair operators, dynamically learning which operators perform best during the search process.

- N : number of customers
- $c_{i,j}$: the travel time from point i to point j
- e_i, l_i, d_i : the earliest time, latest time, and service duration at point i
- q : the destruction degree, representing the number of nodes removed per iteration
- T : the current temperature controlling the acceptance probability
- α : the cooling rate for the temperature
- w_d, w_r : the dynamic weights associated with the destroy and repair operators

4.2.1 Route Evaluation

This ALNS implementation allows temporary exploration through infeasible temporal space to bridge disconnected feasible regions. To enforce final feasibility, time-window violations are heavily penalized. The objective function $f(s)$ for a given route sequence s is evaluated as:

$$f(s) = \sum c_{u,v} + M \cdot \sum \max(0, t_v - l_v)$$

where t_v is the actual arrival time at node v , and M is a massive penalty scalar used to heavily discourage late arrivals.

4.2.2 Destroy and Repair Operators

At each iteration, the current route is perturbed using one destroy and one repair operator.

(1) **Destroy Operators:** These heuristics remove q nodes from the current route.

- **Random Removal:** Selects q nodes at random without replacement.
- **Segment Removal:** Removes a contiguous block of q nodes.
- **Worst Removal:** Targets nodes that contribute the most to the routing cost. The cost contribution Δc_i of a node i positioned between $prev$ and $next$ is calculated as:

$$\Delta c_i = c_{prev,i} + c_{i,next} - c_{prev,next}$$

The q nodes with the highest Δc_i values are removed from the route.

(2) **Repair Operators:**

To find better routes, the algorithm sometimes needs to temporarily break the rules (arriving late) to connect two good paths. To make sure the final answer is actually valid, it uses a “soft penalty” system that heavily fines any lateness. When inserting a new customer i into position k (between a $prev$ customer and a $next$ customer), the algorithm calculates the lateness penalty:

- **Customer i Lateness:** $viol_i = \max(0, arr_i - l_i)$
- **Next Customer Lateness:** $viol_{next} = \max(0, arr_{next} - l_{next})$
- **Total Time Penalty:** $P_{time} = (viol_i + viol_{next}) \cdot N$ (Multiplied by the total number of customers N to make lateness much more expensive than driving distance).

The total objective cost to insert the customer is the extra driving distance plus this time penalty:

$$\Delta f(i, k) = c_{prev,i} + c_{i,next} - c_{prev,next} + P_{time}$$

The repair operators use this cost formula to rebuild the route:

- **Greedy Insertion:** Checks every unassigned customer and inserts the one with the lowest $\Delta f(i, k)$.
- **Regret-2 Insertion:** It calculates regret—the cost difference between a customer's best spot and second-best spot and prioritizes inserting the customer with the highest regret first.

$$regret_i = \Delta f(i, k_2) - \Delta f(i, k_1)$$

- **Random Insertion:** Insert customers into random spots.

4.2.3 Adaptive Weight Adjustment

The selection of operators is controlled by a Roulette Wheel mechanism. The probability P_i of choosing an operator i is proportional to its current weight w_i :

$$P_i = \frac{w_i}{\sum_k w_k}$$

After an iteration, the applied operators are scored based on the outcome of the newly generated route:

- $\sigma = 33$: The route is a new global best.
- $\sigma = 13$: The route is non-improving but accepted via Simulated Annealing criterion.
- $\sigma = 9$: The route is better than the current route.
- $\sigma = 0$: The route is rejected.

The weights are then updated using a decay factor $\lambda \in [0, 1]$ to ensure the algorithm retains a memory of past performance while reacting to the current phase of the search:

$$w_i \leftarrow \lambda \cdot w_i + (1 - \lambda) \cdot \sigma$$

4.2.4 Simulated Annealing Acceptance Criterion

To maintain diversity and avoid premature convergence on local optima, the algorithm employs a Simulated Annealing acceptance criterion. A newly generated route s' with cost $f(s')$ that is strictly worse than the current route s is accepted with probability:

$$P(\text{accept}) = \exp\left(-\frac{f(s') - f(s)}{T}\right)$$

The temperature T is geometrically reduced at the end of each iteration via the cooling rate α ($T \leftarrow \alpha \cdot T$), gradually shifting the algorithm from exploration to exploitation.

Algorithm 5: Adaptive Large Neighborhood Search (ALNS)

Input: $N, e, l, d, c, \text{max_no_imp}$
Output: Improved feasible route

```

1 Function ALNS( $N, e, l, d, c, \text{max\_no\_imp}$ ):
2    $\text{best\_route}, \text{best\_cost} \leftarrow \text{greedy}(N, e, l, d, c)$   $q \leftarrow \max(5, \lfloor N \cdot 0.15 \rfloor)$ 
3    $T \leftarrow -0.1 \cdot \text{best\_cost} / \ln(0.6)$ ,  $\alpha \leftarrow 0.995$ ,  $\lambda \leftarrow 0.8$  // See appendix 4
4    $\sigma_1 \leftarrow 33$ ,  $\sigma_2 \leftarrow 9$ ,  $\sigma_3 \leftarrow 13$ 
5    $w_d[1 \dots 3] \leftarrow 1$ ,  $w_r[1 \dots 3] \leftarrow 1$ 
6   while not_improved < max_no_imp do
7      $P_d \leftarrow w_d / \sum w_d$  // Destroy operator probabilities
8      $P_r \leftarrow w_r / \sum w_r$  // Repair operator probabilities
9      $d\_idx \leftarrow \text{RouletteWheel}(P_d)$ 
10     $r\_idx \leftarrow \text{RouletteWheel}(P_r)$ 
11     $\text{temp\_route}, \text{removed} \leftarrow \text{Destroy}(\text{current\_route}, q, d\_idx)$ 
12     $\text{new\_route} \leftarrow \text{Repair}(\text{temp\_route}, \text{removed}, r\_idx)$ 
13     $\text{new\_cost} \leftarrow \text{Evaluate}(\text{new\_route})$ 
14     $\text{score} \leftarrow 0$ 
15    if new_cost < best_cost then
16       $\text{best\_route} \leftarrow \text{new\_route}$ ,  $\text{best\_cost} \leftarrow \text{new\_cost}$ 
17       $\text{current\_route} \leftarrow \text{new\_route}$ ,  $\text{current\_cost} \leftarrow \text{new\_cost}$ 
18       $\text{score} \leftarrow \sigma_1$ 
19      not_improved  $\leftarrow 0$ 
20    else if new_cost < current_cost then
21       $\text{current\_route} \leftarrow \text{new\_route}$ ,  $\text{current\_cost} \leftarrow \text{new\_cost}$ 
22       $\text{score} \leftarrow \sigma_2$ 
23      not_improved  $\leftarrow \text{not\_improved} + 1$ 
24    else
25       $p \leftarrow \exp(-\frac{\text{new\_cost} - \text{current\_cost}}{T})$ 
26      if Random(0, 1) <  $p$  then
27         $\text{current\_route} \leftarrow \text{new\_route}$ ,  $\text{current\_cost} \leftarrow \text{new\_cost}$ 
28         $\text{score} \leftarrow \sigma_3$ 
29      not_improved  $\leftarrow \text{not\_improved} + 1$ 
30     $w_d[d\_idx] \leftarrow \lambda \cdot w_d[d\_idx] + (1 - \lambda) \cdot \text{score}$ 
31     $w_r[r\_idx] \leftarrow \lambda \cdot w_r[r\_idx] + (1 - \lambda) \cdot \text{score}$ 
32     $T \leftarrow \alpha \cdot T$  // Temperature cooling
33  return best_route, best_cost

```

5. Results and Analysis

Note: All test cases were done on a plugged-in Macbook Air M4.

5.1 Hustack test cases (Tight time windows)

N	Branch & Bound		CP		MIP		Greedy		Local Search		ACO		ALNS	
	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)
5	380	0.37	380	0.41	380	0.47	380	0.38	380	0.29	380	0.31	380	0.46
10	620	0.28	620	0.28	620	0.30	620	0.26	620	0.28	620	0.36	620	0.37
100	TLE	TLE	56310	0.32	56310	0.38	56310	0.26	56310	0.27	56310	5.15	56310	0.96
200	TLE	TLE	98410	0.49	98410	0.69	98410	0.27	98410	0.27	98410	10.73	98410	1.69
300	TLE	TLE	160090	0.87	160090	1.23	160090	0.29	160090	0.30	160090	16.61	160090	3.50
500	TLE	TLE	259360	2.03	259360	3.26	259360	0.32	259360	0.45	259360	31.59	259360	10.66
600	TLE	TLE	326720	2.84	326720	4.79	326720	0.31	326720	0.55	326720	41.06	326720	15.03
700	TLE	TLE	357980	3.98	357980	6.44	357980	0.32	357980	0.63	357980	56.30	357980	21.17
900	TLE	TLE	454330	7.29	454330	11.06	454330	0.36	454330	0.98	454330	74.12	454330	36.81
1000	TLE	TLE	510850	9.68	510850	14.54	510850	0.40	510850	1.22	510850	87.33	510850	43.98

Table 1: Cost and Runtime of Algorithms (Hustack test cases)

5.1.1 Overview

- Branch and Bound failed to compute anything beyond $N = 10$.
- Surprisingly, the remaining algorithms all managed to achieve optimality in a reasonable time, with Greedy being the fastest and ACO being the slowest.
- However, these results do not illustrate the power of Local Search and metaheuristic framework, as they all rely on Greedy to initialize a route (which in these test cases, were the optimal route).

5.1.2 Time Analysis

- Greedy is the fastest algorithm overall, needing under half a second to find the solution even for the largest test case.
- Because of the tight time windows, exact solvers like CP and MIP were quick to find a route and prove its optimality.
- Metaheuristic exploration nature makes ACO and ALNS run slower, but they struggle to find an alternative path due to the tight time constraints.

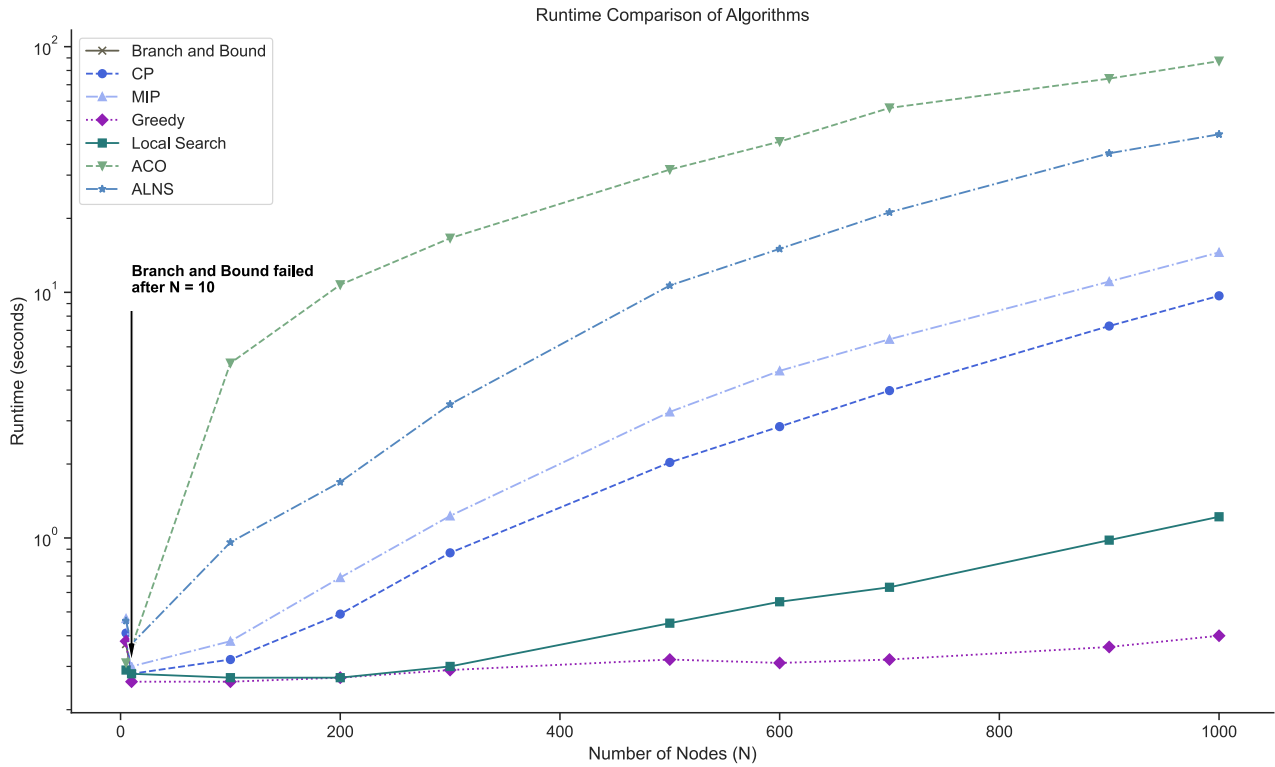


Figure 1: Runtime comparison of Algorithms (Hustack test cases)

5.1.3 Cost Analysis

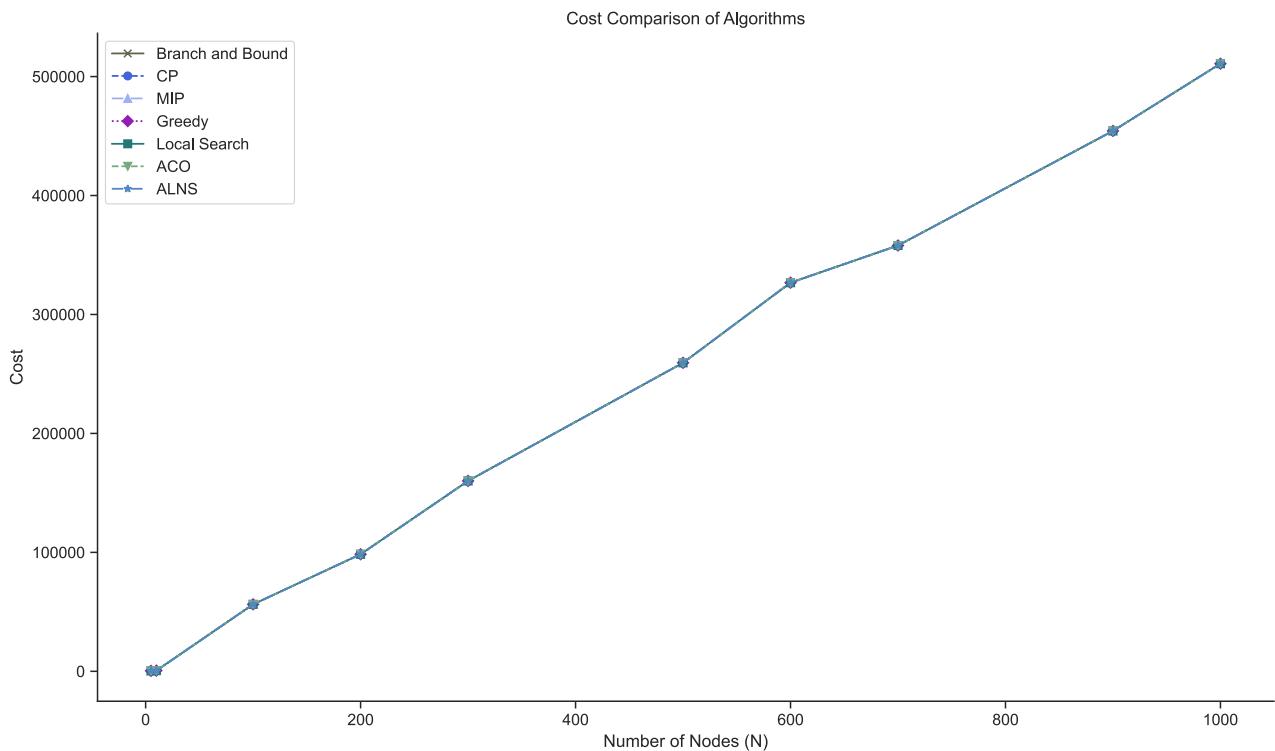


Figure 2: Cost comparison of Algorithms (Hustack test cases)

- All algorithms (except for Branch and Bound) managed to find the optimal cost.
- It should be noted that ACO and ALNS rely on Greedy to initialize a route. If a different method is used to initialize a route (i.e sort the node by e_i), it may not converge to the optimal solution.

5.1.4 Conclusion

- In these tight time constraints test case, finding a feasible route alone is difficult. Therefore, the only feasible route is usually the optimal route.
- Greedy is the best algorithm here due to it lightning-fast ability to find a feasible route.
- If you want assurance on optimality, consider exact solvers like MIP and CP, as the tight constraints can massively reduce the search space, enabling them to find the optimal route in a reasonable amount of time.

5.2 Code-generated test cases (Loose time windows)

These test cases were deliberately configured with loose and overlapping time windows to research the exploration capabilities of Local Search and metaheuristic frameworks.

N	Branch & Bound		CP		MIP		Greedy		Local Search		ACO		ALNS	
	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)
5	2371	0.37	2371	0.39	2371	0.53	2377	0.39	2371	0.38	2371	0.40	2371	0.42
10	4688	0.32	4688	0.28	4688	0.33	5211	0.28	4688	0.28	4688	0.37	4688	0.33
50	TLE	TLE	23954	0.36	23954	0.99	28205	0.30	24842	0.28	24935	7.36	23954	0.46
100	TLE	TLE	46420	0.99	46420	16.60	53601	0.32	46991	0.33	49393	26.74	46430	0.81
150	TLE	TLE	74608	1.62	74608	10.07	85747	0.28	76277	0.46	82911	26.57	74619	1.12
200	TLE	TLE	92052	4.37	92052	73.56	104698	0.27	92796	0.77	102388	38.58	92308	2.10
250	TLE	TLE	108783	7.51	108784	49.42	124212	0.28	109857	1.26	121002	91.72	108783	4.58
300	TLE	TLE	136141	9.98	TLE	TLE	155882	0.28	137870	1.99	153174	88.26	136220	5.46
350	TLE	TLE	159677	16.93	TLE	TLE	186386	0.31	162434	4.59	185605	34.90	159803	14.93
400	TLE	TLE	186736	17.87	TLE	TLE	213093	0.30	189030	5.94	212152	38.90	187107	16.14
450	TLE	TLE	204739	45.91	TLE	TLE	234134	0.28	209082	10.40	232369	63.82	234134	8.35
500	TLE	TLE	238215	54.04	TLE	TLE	272086	0.31	241260	15.36	271121	35.15	245630	36.04
550	TLE	TLE	237693	75.23	TLE	TLE	281512	0.34	242040	22.25	280723	91.69	275301	13.13
600	TLE	TLE	266592	63.78	TLE	TLE	308278	0.33	270277	32.36	306616	164.54	299872	16.23
650	TLE	TLE	294853	78.68	TLE	TLE	338656	0.32	298798	41.29	336451	274.58	329576	17.71
700	TLE	TLE	328487	89.39	TLE	TLE	380265	0.33	331877	42.62	379000	186.87	380265	15.24
750	TLE	TLE	345919	117.59	TLE	TLE	401122	0.34	351847	59.35	401095	105.35	401122	25.24
800	TLE	TLE	364389	202.78	TLE	TLE	414980	0.37	369860	71.91	414973	70.56	409022	25.04
850	TLE	TLE	381481	175.77	TLE	TLE	441387	0.38	386017	96.64	441078	72.29	434408	19.77
900	TLE	TLE	410156	163.16	TLE	TLE	472303	0.37	413617	104.99	471519	83.56	443597	64.80
950	TLE	TLE	438158	244.80	TLE	TLE	505810	0.41	444472	129.28	505810	70.26	505810	33.39
1000	TLE	TLE	450133	225.18	TLE	TLE	517615	0.42	455099	151.09	517292	149.80	488484	78.38

Table 2: Cost and Runtime of Algorithms (Code-generated test cases)

5.2.1 Overview

- Branch and Bound hits an exponential wall almost instantly, failing to compute anything beyond $N = 10$. MIP performs slightly better but failed after $N = 250$.
- Interestingly, the Constraint Programming (CP) model manages to survive the combinatorial explosion, solving the $N = 1000$ network in roughly 225 seconds. This provides a rare, complete baseline of optimality across all problem sizes.

5.2.2 Time Analysis

- Greedy remains virtually flat against the bottom of the axis. Even at $N = 1000$, it computes a route in 0.42 seconds, proving it is the only viable option for strictly real-time applications.
- CP is the slowest, scaling up to 225 seconds. Local Search and ACO follow a similar upper-bound trajectory, both taking around 150 seconds to complete the 1000-node network.
- ALNS demonstrates phenomenal time scalability. At $N = 1000$, it finishes in just 78 seconds—nearly twice as fast as Local Search and ACO, and three times faster than CP.

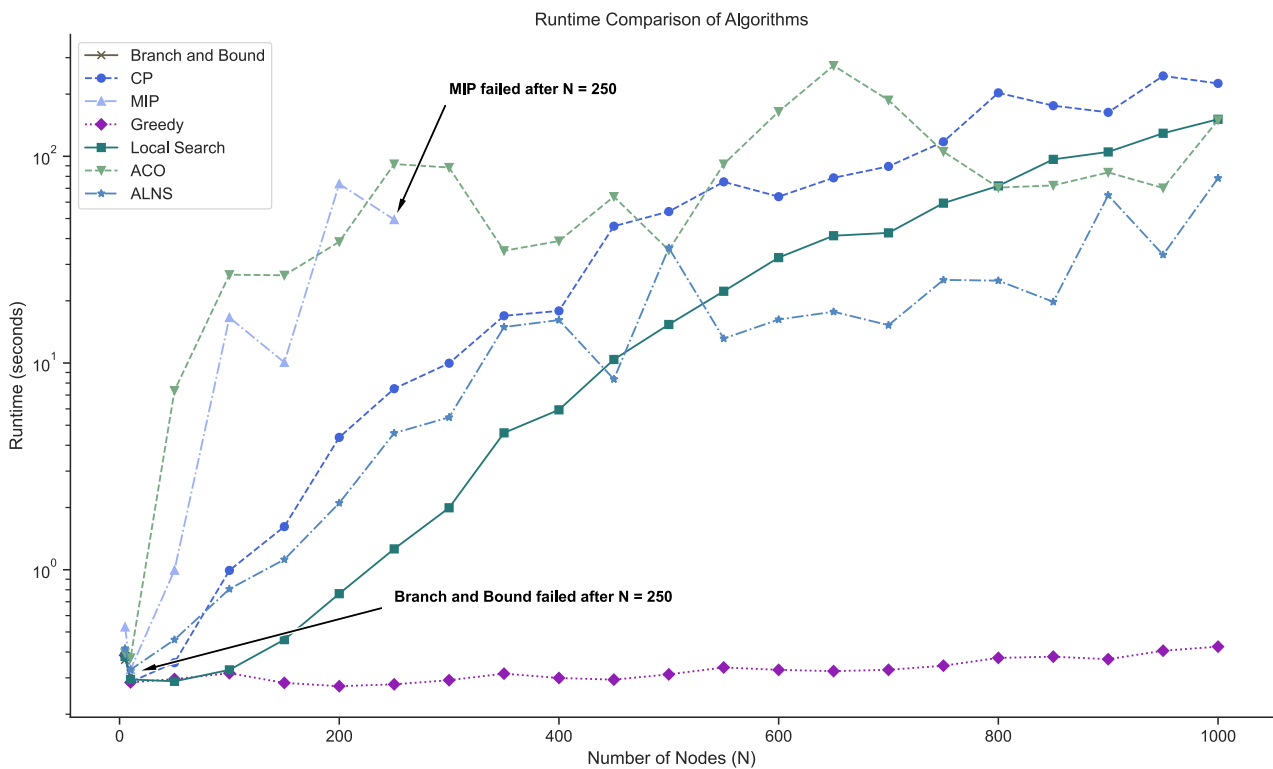


Figure 3: Runtime comparison of Algorithms (Code-generated test cases)

5.2.3 Cost Analysis

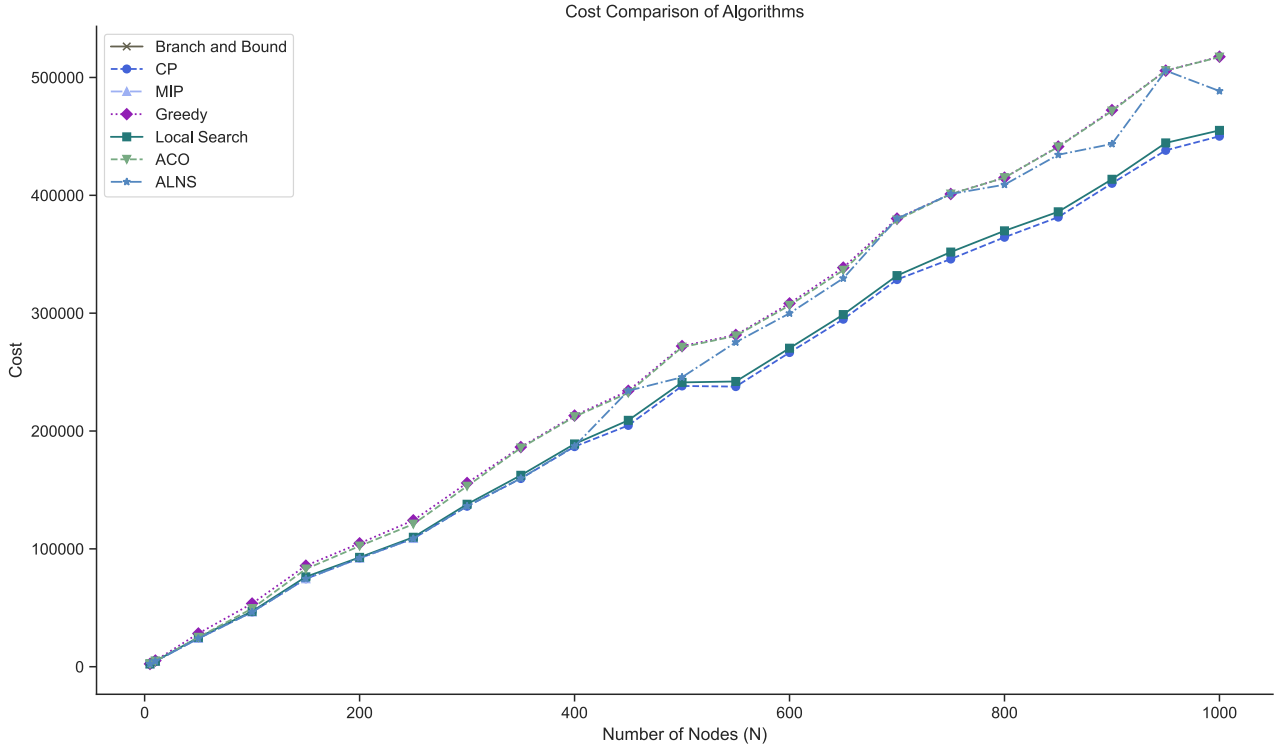


Figure 4: Cost comparison of Algorithms (Code-generated test cases)

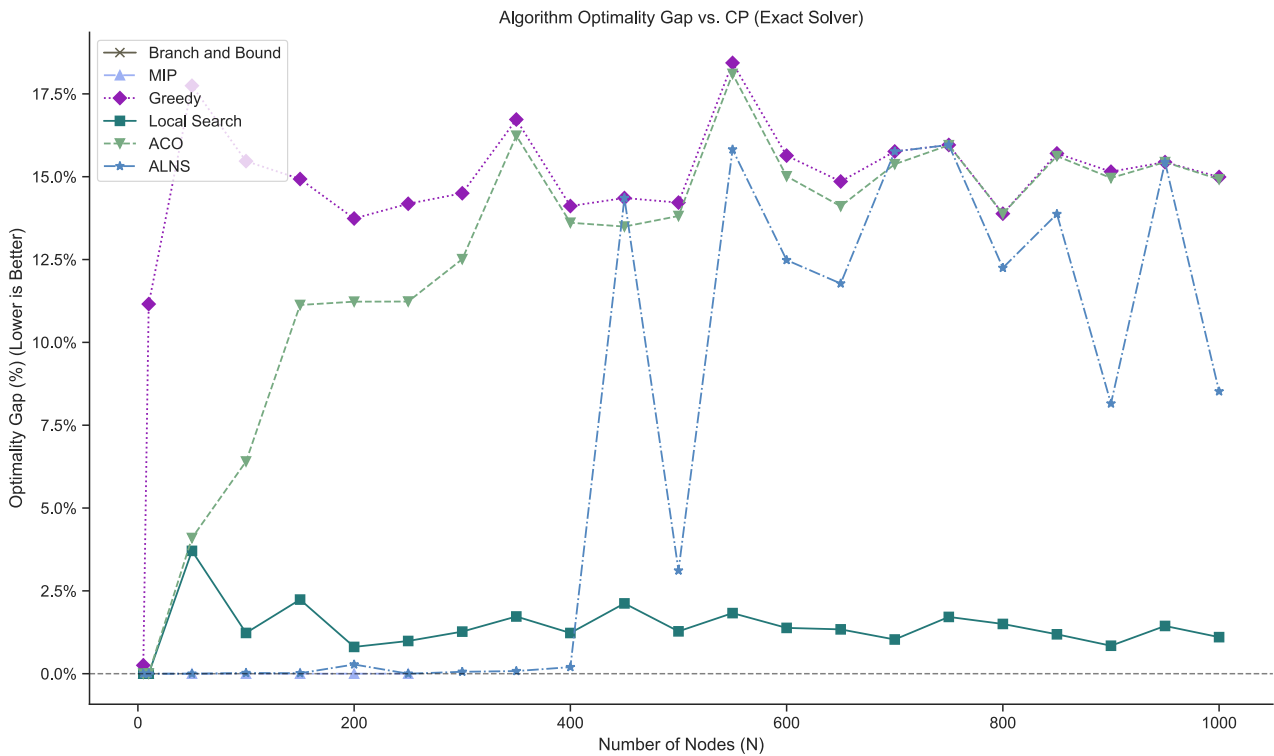


Figure 5: Optimality Gap of Algorithms vs. CP Baseline (Code-generated test cases)

- The cost comparison graph shows two distinct lines: an upper-bound cost setted by Greedy and a lower-bound cost achieved by CP.
- At small $N \leq 400$, ALNS displays a 0% optimality gap while running faster than CP. However, it fluctuates significantly with increasing N , landing it in the middle.
- ACO fails to provide value at large scale. With $N = 1000$, it takes 150 seconds only to return a cost of 517,292, which is virtually indistinguishable from the instant Greedy route and about 15% higher than the optimal route.
- Local Search's results is incredibly close to the optimal results, staying under 2% deviation from CP for the majority of the graph.

5.2.4 Conclusion

- If you need a mathematically perfect answer, CP is remarkably reliable up to 1,000 nodes, though it takes almost four minutes (225.18 seconds) to finish the largest dataset.
- If you want a near-perfect answer faster, Local Search is your best bet; it delivers an incredibly accurate route (costing 455,099 compared to the perfect 450,133) and saves over a minute of processing time (151.09 seconds).
- ALNS represents a balance between computation time and quality, but its performance varies drastically for large N .
- Greedy is useful if you absolutely need an answer instantly (less than 0.5 seconds for all test cases), but you must be willing to accept a highly inefficient route.
- Branch and Bound, MIP and ACO should be avoided due to their low performance. It should be noted that ACO performance could be improved by optimizing its parameters.

6. Conclusion

6.1 Strengths and Limitations of Each Algorithm

- **Branch and Bound:** Its primary strength is guaranteeing an optimal route. However, its major limitation is its exponential time complexity; it hits an exponential wall almost immediately and fails to compute instances beyond $N = 10$.
- **Mixed Integer Programming (MIP):** MIP performs slightly better than Branch and Bound in handling the combinatorial explosion. Despite this, it still ultimately fails on larger code-generated instances after $N = 250$.
- **Constraint Programming (CP):** CP is an outstanding and remarkably reliable exact solver, capable of finding mathematically perfect answers for datasets scaling up to 1,000 nodes. Its limitation is computation time, as it is the slowest algorithm and takes nearly four minutes (225.18 seconds) to process the largest datasets.
- **Greedy Algorithm:** The Greedy algorithm is unmatched in speed, computing routes in under 0.5 seconds even for 1,000 nodes. It is the optimal choice for real-time applications or scenarios with very tight time constraints, where simply finding a feasible route equates to finding the optimal one. Its critical limitation is that it produces highly inefficient, suboptimal routes when time windows are loose.
- **Local Search:** This algorithm provides an excellent balance between computation time and solution quality. It delivers incredibly accurate routes that stay under a 2% deviation from the perfect baseline, while saving over a minute of processing time compared to exact solvers like CP.
- **Ant Colony Optimization (ACO):** ACO struggles to provide value at large scales. On large networks, it takes considerable time (around 150 seconds) only to return costs roughly 15% higher than the optimal route. It was also the slowest algorithm tested under tight constraints.
- **Adaptive Large Neighborhood Search (ALNS):** ALNS features phenomenal time scalability, finishing large networks almost twice as fast as Local Search and ACO. However, its major limitation lies in consistency, as its performance and solution optimality fluctuate drastically as the number of nodes increases.

6.2 Future Research Directions

- **Parameter Optimization:** Since ACO demonstrated generally low performance across the test cases, future work could focus on systematically optimizing its parameters. Fine-tuning variables like pheromone importance and evaporation rates could potentially improve its exploration capabilities.
- **Exploration of Alternative Algorithms:** While this study focused on ACO and ALNS, future research should investigate the efficacy of other well-established algorithms. Exploring methods such as standalone Simulated Annealing (SA), Tabu Search, or Genetic Algorithms could provide further insights into balancing the trade-off between computational efficiency and solution optimality.

7. Appendix

7.1 Appendix 1. Derivation of $M_{i,j}$

In the MIP constraints section, we have this time constraint:

$$t_j \geq t_i + d_i + c_{i,j} - M_{i,j}(1 - x_{i,j}) \quad \forall (i, j) \in A \text{ and } j \neq 0$$

where

$$M_{i,j} = \max(0, l_i - e_j + d_i + c_{i,j})$$

The arrival time at point j must be greater than or equal to the earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$).

- **Case 1.** $x_{i,j} = 1$

$$t_j \geq t_i + d_i + c_{i,j}$$

When the edge (i, j) is active, this is the time constraint we have to satisfy.

- **Case 2.** $x_{i,j} = 0$

$$t_j \geq t_i + d_i + c_{i,j} - M_{i,j}$$

$$M_{i,j} \geq t_i + d_i + c_{i,j} - t_j$$

Because $M_{i,j}$ always has to satisfy the inequality, we consider the worst case when the right-hand side is maximal, which occurs when $t_i = l_i$ and $t_j = e_j$

$$M_{i,j} \geq l_i + d_i + c_{i,j} - e_j$$

We consider the smallest M that satisfies the inequality (to reduce the search space for MIP):

$$M_{i,j} = l_i + d_i + c_{i,j} - e_j$$

If M is negative, $-M_{i,j}$ would be positive, which would tighten our constraint of t_j while (i, j) is non-active.

Because of that, we take the maximum:

$$M_{i,j} = \max(0, l_i + d_i + c_{i,j} - e_j)$$

7.2 Appendix 2. Explanation of MTZ constraint

In the MIP constraints section, we have this constraint:

$$u_i - u_j + N \cdot x_{i,j} \leq N - 1$$

- **Case 1.** $x_{i,j} = 1$

$$u_j - u_i + N \leq N - 1$$

$$u_j \geq u_i + 1$$

If edge (i, j) is active, then the sequencing index u_j must be larger than u_i . This is the constraint we want.

- **Case 2.** $x_{i,j} = 0$

$$u_j - u_i \leq N - 1$$

In the worst case: $u_j = N$, $u_i = 1$

$$N - 1 \leq N - 1$$

Therefore, when edge (i, j) is not active, this constraint is always satisfied (which means it is "turned off")

7.3 Appendix 3. Derivation of $\tau_{\max}$

In MMAS, only one ant (the iteration-best ant) is allowed to deposit pheromone at the end of each iteration. The core idea behind $\tau_{\max}$ is: If an edge is consistently getting picked, what is the asymptotic upper-bound of pheromone on that edge?

We have this recurrence relation:

$$\tau_{i,j}(t+1) = (1 - \rho)\tau_{i,j}(t) + \frac{Q}{L_{best}}$$

As $t \rightarrow \infty$, $\tau_{i,j} \rightarrow \tau_{\max}$:

$$\tau_{\max} = (1 - \rho)\tau_{\max} + \frac{Q}{L_{best}}$$

$$\rho\tau_{\max} = \frac{Q}{L_{best}}$$

$$\tau_{\max} = \frac{Q}{\rho \cdot L_{best}}$$

Therefore, $\tau_{\max} = \frac{Q}{\rho \cdot L_{best}}$.

7.4 Appendix 4. Derivation of T

Initially, T is configured such that the algorithm selects a 10% worse route 60% of the time. In the code, we have:

$$T = \frac{-0.1 \cdot \text{best_cost}}{\ln 0.6}$$

To derive this, consider the Metropolis criterion:

$$P = e^{\frac{-\Delta E}{T}}$$

A 10% worse route means that $\Delta E = 0.1 \cdot \text{best_cost}$. A 60% acceptance rate means that $P = 60\%$.

$$0.6 = e^{\frac{-0.1 \cdot \text{best_cost}}{T}}$$

$$\ln 0.6 = \frac{-0.1 \cdot \text{best_cost}}{T}$$

$$T = \frac{-0.1 \cdot \text{best_cost}}{\ln 0.6}$$